



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



GraphQL на Java



Фреймворк для GraphQL

- GraphQL является спецификацией, по сути языком запросов для API.
- Для внедрения GraphQL на сервере мы обычно используем фреймворки.
- В рамках нашей лабораторной работы мы опробуем Domain Graph Service, фреймворк, созданный командой Netflix, который позволяет использовать GraphQL в приложении Spring Boot подключив всего несколько зависимостей.



Фреймворк для GraphQL

The screenshot shows the 'Getting Started' page of the DGS Framework. The page has a blue header with the DGS logo, GitHub repository information (v11.1.0, 3.3k stars, 331 forks), and a search bar. A left sidebar lists navigation links: DGS Framework, Getting Started (highlighted), Spring GraphQL Integration, New and Noteworthy, Configuration, Data fetching, Testing, Mutations, Code Generation, Data Loaders (N+1), Error Handling, Federation, Examples, Videos, and Advanced. The main content area is titled 'Getting Started' and 'Introduction'. It explains that the DGS framework makes it easy to create GraphQL services with Spring Boot and provides an easy-to-use annotation-based programming model. It also mentions that the framework is primarily maintained by Netflix. A section titled 'Create a new Spring Boot application' is partially visible, stating that the framework is now based on Spring Boot 3 and that the easiest way to create a DGS project is to use the Spring Initializr. A right sidebar contains a 'Table of contents' with links to Introduction, Create a new Spring Boot application, Requirements, Adding the DGS Framework dependency with Spring for GraphQL, Creating a Schema, Implement a Data Fetcher, Test the app with GraphQL, Install the IntelliJ plugin, and Next steps.

DGS Framework

Getting Started

Spring GraphQL Integration

New and Noteworthy

Configuration

Data fetching

Testing

Mutations

Code Generation

Data Loaders (N+1)

Error Handling

Federation

Examples

Videos

Advanced

Getting Started

Introduction

The DGS framework makes it easy to create GraphQL services with Spring Boot. The framework provides an easy-to-use annotation based programming model, and all the advanced features needed to build and run GraphQL services at scale.

The DGS framework is primarily maintained by Netflix, surrounded by an active community. At Netflix we build our GraphQL architecture on the DGS framework.

Create a new Spring Boot application

The DGS framework is now based on Spring Boot 3

The easiest way to create a DGS project is to use the Spring Initializr. You'll need to following dependencies:

1. Netflix DGS

Table of contents

- Introduction
- Create a new Spring Boot application
- Requirements
- Adding the DGS Framework dependency with Spring for GraphQL
- Creating a Schema
- Implement a Data Fetcher
- Test the app with GraphQL
- Install the IntelliJ plugin
- Next steps



Фреймворк для GraphQL

- Разработчиками сделан starter, который позволит просто создать проект



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Java ☐ Kotlin ☐ Groovy

☒ Maven

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (M4) ☒ 4.0.6 (SNAPSHOT) ☐ 4.0.5

☐ 3.5.14 (SNAPSHOT) ☐ 3.5.13

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☒ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Netflix DGS WEB

Build GraphQL applications with Netflix DGS and Spring for GraphQL.

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.



Фреймворк для GraphQL

- Если хотите добавить стартер в свой проект

```
<dependencies>
  <dependency>
    <groupId>com.netflix.graphql.dgs</groupId>

    <artifactId>graphql-dgs-spring-graphql-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>com.netflix.graphql.dgs</groupId>

    <artifactId>graphql-dgs-spring-graphql-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```



Этапы реализации (DGS)

- 1. Описать схему (SDL)
 - `src/main/resources/schema/*.graphqls`
- 2. Реализовать Data Fetcher'ы
 - `@DgsComponent + @DgsQuery / @DgsMutation`
- 3. Настроить Spring Boot
 - Минимальная конфигурация в `application.yml`



schema.graphqls

- Domain Graph Service построен на подходе, когда в начале мы описываем контракт, то есть типы данных, которые могут быть отданы клиентской стороне
- Для этого данный фреймворк проверяет директорию проекта:

`src/main/resources/
schema`

- Давайте в данной директории создадим файл - schema.graphqls



schema.graphqls

```
type Query {  
  shows(titleFilter: String):  
  [Show]  
  show(id: ID!): Show  
}  
type Mutation {  
  addShow(show: SubmittedShow!):  
  Show  
  deleteShow(id: ID!): Boolean  
}  
type Show {  
  id: ID!  
  title: String!  
  releaseYear: Int!  
  genre: Genre  
  rating: Float
```

```
input SubmittedShow  
{  
  title: String!  
  releaseYear:  
  Int!  
  genre: Genre  
}  
  
enum Genre {  
  DRAMA  
  COMEDY  
  THRILLER  
  SCIFI  
  DOCUMENTARY  
}
```



Модель данных

Модель данных в демо проекте – records, но если используется БД и JPA, используйте классы

```
public record  
Show(  
    String id,  
    String title,  
    int  
releaseYear,  
    Genre genre,  
    Double rating  
) {}
```

```
public enum Genre  
{  
    DRAMA,  
    COMEDY,  
    THRILLER,  
    SCIFI,  
    DOCUMENTARY  
}
```

```
public record  
SubmittedShow(  
    String title,  
    int releaseYear,  
    Genre genre  
) {}
```



Data Fetcher

- Data Fetcher отвечают за возврат данных для запроса, они же применяются и в Spring GraphQL
 - Для концепции GraphQL они считаются resolver
- Создадим директорию datafetchers и класс - ShowsDataFetcher



Data Fetcher

```
@DgsComponent
public class ShowsDataFetcher {
    private final ShowService showService;
    public ShowsDataFetcher(ShowService showService) {
        this.showService = showService;
    }
    @DgsQuery
    public List<Show> shows(@InputArgument String
titleFilter) {
        if (titleFilter == null) {
            return showService.findAll();
        }
        return
showService.findByTitleContaining(titleFilter);
    }
    @DgsQuery
    public Show show(@InputArgument String id) {
        return showService.findById(id).orElse(null);
    }
}
```



Объяснение

- Data Fetcher это resolver а значит он должен решить от куда брать данные, обычно он обращается к сервисам!
- В ваших проектах стоит сделать сервисы, которые могут ходить к репозиториям.
- В демонстрационном примере от DGS используется статический список для демонстрации



Объяснение

Аннотация	Описание
@DgsComponent	Регистрирует класс как GraphQL-компонент (специализированный @Component)
@DgsQuery	Метод является resolver'ом для type Query
@DgsMutation	Метод является resolver'ом для type Mutation
@DgsSubscription	Метод является resolver'ом для type Subscription
@InputArgument	Маппинг аргумента из GraphQL-запроса



Data Fetcher

```
@DgsComponent
public class ShowsMutation {
    private final ShowService showService;
    public ShowsMutation(ShowService showService) {
        this.showService = showService;
    }
    @DgsMutation
    public Show addShow(@InputArgument SubmittedShow show)
    {
        return showService.createShow(
            show.title(),
            show.releaseYear(),
            show.genre(),
            null
        );
    }
    @DgsMutation
    public boolean deleteShow(@InputArgument String id) {
        return showService.deleteById(id);
    }
}
```



Конфигурация

- application.yml

```
spring:
  graphql:
    graphiql:
      enabled: true
      path: /graphiql
    schema:
      locations:
        classpath:schema/
      introspection:
        enabled: true
  server:
    port: 8080
```



GraphiQL

- Протестировать все запросы можно в среде GraphiQL – интерфейс запросов для тестирования API
- Зачастую данный интерфейс доступен из коробки запускается при старте приложения
- Запустите проект и перейдите по ссылке:
- <http://localhost:8080/graphiql>
- Но можно и с помощью REST плагинов



GraphiQL

Docs

🔍 Ctrl Alt |

A GraphQL schema provides a root type for each kind of operation.

🔍 Root Types

query: Query

mutation: Mutation

🔍 All Schema Types

Boolean

ErrorDetail

ErrorType

Float

Genre

ID

Int

Show

String

SubmittedShow

_FieldSet

_Service

<untitled>

+

GraphiQL

1



Variables

Headers



1



Тестируем

```
query {  
  shows {  
    id  
    title  
    releaseYear  
    genre  
    rating  
  }  
}
```

```
{  
  "data": {  
    "shows": [  
      {  
        "id": "446ce427-4355-4a98-8174-a1d8857689fb",  
        "title": "The Crown",  
        "releaseYear": 2016,  
        "genre": "DRAMA",  
        "rating": 8.6  
      },  
      {  
        "id": "af1177fb-6940-4b09-8b1f-29b93ac26157",  
        "title": "Stranger Things",  
        "releaseYear": 2016,  
        "genre": "SCIFI",  
        "rating": 8.7  
      },  
      {  
        "id": "d5753390-ccd5-4cd9-8207-5d204b3836c6",  
        "title": "Ozark",  
        "releaseYear": 2017,  
        "genre": "DRAMA",  
        "rating": 8.5  
      },  
      {  
        "id": "cfb18aec-d58c-4c9a-8634-8085a4d6155f",  
        "title": "Clarkson's Farm",  
        "releaseYear": 2021,  
        "genre": "COMEDY",  
        "rating": 9  
      }  
    ]  
  }  
}
```



Тестируем

```
query {  
  shows(titleFilter: "Stranger") {  
    title  
    releaseYear  
  }  
}
```

```
{  
  "data": {  
    "shows": [  
      {  
        "title": "Stranger Things",  
        "releaseYear": 2016  
      }  
    ]  
  }  
}
```



Тестируем

```
query {  
  show(id: "446ce427-4355-4a98-8174-a1d8857689fb") {  
    title  
    rating  
  }  
}
```

```
{  
  "data": {  
    "show": {  
      "title": "The Crown",  
      "rating": 8.6  
    }  
  }  
}
```



Тестируем

```
mutation {  
  addShow(show: {title: "Black Mirror", releaseYear: 2011, genre: SCIFI}) {  
    id  
    title  
    releaseYear  
    genre  
  }  
}
```

```
{  
  "data": {  
    "addShow": {  
      "id": "ceab02f2-e8d9-4fdb-8fe3-afbac0f73fb1",  
      "title": "Black Mirror",  
      "releaseYear": 2011,  
      "genre": "SCIFI"  
    }  
  }  
}
```



Тестируем

```
mutation {  
  deleteShow(id: "87a3cf0e-0e6c-4bbb-870d-6297b470b478")  
}
```

```
{  
  "data": {  
    "deleteShow": false  
  }  
}
```



N+1 проблема

- При запросе списка с вложенными полями резолвер вызывается N+1 раз

```
query {  
  shows {  
    title  
    director {  
      name  
    }  
  }  
}
```



N+1 проблема

- При запросе списка с вложенными полями резолвер вызывается N+1 раз

```
query {  
  shows {  
    title  
    director {  
      name  
    }  
  }  
}
```



DataLoader (batch loading)

```
@DgsDataLoader(name = "directors")
public class DirectorsDataLoader implements BatchLoader<String, Director> {

    private final DirectorService directorService;
    public DirectorsDataLoader(DirectorService directorService) {
        this.directorService = directorService;
    }

    @Override
    public CompletionStage<List<Director>> load(List<String> directorIds) {
        return CompletableFuture.supplyAsync(() ->
            directorService.findByIds(directorIds));
    }
}
```



DataLoader (batch loading)

```
@DgsData(parentType = "Show", field = "director")
public CompletableFuture<Director> director(DgsDataFetchingEnvironment
dfe) {
    DataLoader<String, Director> loader =
dfe.getDataLoader("directors");
    Show show = dfe.getSource();
    if (show.directorId() == null) {
        return CompletableFuture.completedFuture(null);
    }
    return loader.load(show.directorId());
}
```



Обработка ошибок

В GraphQL ошибки возвращаются вместе с данными (в отличие от REST, где ошибка заменяет весь ответ)

```
query {  
  show(id: "999") {  
    title  
    rating  
  }  
}
```

```
{  
  "errors": [  
    {  
      "message": "Show not found: 999",  
      "locations": [  
        {  
          "line": 1,  
          "column": 9  
        }  
      ],  
      "path": [  
        "show"  
      ],  
      "extensions": {  
        "errorType": "UNAVAILABLE",  
        "errorDetail": "SERVICE_ERROR"  
      }  
    }  
  ],  
  "data": {  
    "show": null  
  }  
}
```



Кастомное исключение

```
public class ShowNotFoundException extends  
RuntimeException {  
    public ShowNotFoundException(String id) {  
        super("Show not found: " + id);  
    }  
}
```



Глобальный обработчик

```
@Component
public class CustomDataFetchingExceptionHandler implements
DataFetcherExceptionHandler {

    @Override
    public CompletableFuture<DataFetcherExceptionHandlerResult> handleException(
        DataFetcherExceptionHandlerParameters params) {

        // Логирование, маппинг ошибок

        return CompletableFuture.completedFuture(
DataFetcherExceptionHandlerResult.newResult().error(graphqlError).build()
        );
    }
}
```



Безопасность

```
@Configuration
public class GraphQLSecurityConfig {

    @Bean
    public Instrumentation maxQueryDepthInstrumentation() {
        // Ограничиваем максимальную глубину запроса: 20 уровней
        // (достаточно для GraphQL Introspection, блокирует бесконечную
        рекурсию)
        return new MaxQueryDepthInstrumentation(20);
    }

    @Bean
    public Instrumentation maxQueryComplexityInstrumentation() {
        // Ограничиваем максимальную сложность запроса: 200 единиц
        // (по умолчанию каждое запрашиваемое поле "весит" 1 единицу)
        return new MaxQueryComplexityInstrumentation(200);
    }
}
```



Будут вопросы? Не стесняйтесь!

